



A study of manycore shared Memory Architecture as a way to build SOC applications

Yosi Ben Asher	Yousef Shajrawi, Y. Gendel	Gadi Haber	Oren Segal
CS. Dept.	IBM	Intel	EE dep.
University of Haifa	Haifa Labs	Haifa Matam	UMass Lowell
yosi@cs.haifa.ac.il	yousefs/gendel@il.ibm.com	gadi.haber@intel.com	oren@segals.net

November 22, 2014

Abstract

Manycore shared memory architectures hold a significant premise to speed up and simplify SOC's. Using many homogeneous small-cores will allow replacing the hardware accelerators of SOC's by parallel algorithms communicating through shared memory. Currently shared memory is realized by maintaining cache-consistency across the cores, caching all the connected cores to one main memory module. This approach, though used today, is not likely to be scalable enough to support the high number of cores needed for highly parallel SOC's. Therefore we consider a theoretical scheme for shared memory wherein: the shared address space is divided between a set of memory modules; and a communication network allows each core to access every such module in parallel. Load-balancing between the memory modules is obtained by rehashing the memory address-space. We consider practical aspects involved with a practical realization of this scheme, e.g., how will the wire complexity of the communication network affect the execution time.

We have designed a simple generic shared memory architecture, synthesized it to 2, 4, 8, 16, ..., 1024 – cores for FPGA virtex-7 and evaluated it on several parallel programs. The synthesis results and the execution measurements show that, for the FPGA, all problematic aspects of this construction can be resolved. For example, unlike ASICs, the growing com-

plexity of the communication network is absorbed by the FPGA's routing grid and by its routing mechanism. This makes this type of architectures particularly suitable for FPGAs. We used 32-bits modified PACOBLAZE cores and tested different parameters of this architecture verifying its ability to achieve high speedups. The results suggest that re-hashing is not essential and one hash-function suffice (compared to the family of universal hash functions that is needed by the theoretical construction).¹

1 Introduction

The wide use of SOC designs in low power devices creates a demand for high performance SOC's that comprise of multiple cores executing computations in parallel. In this work we propose a scalable 2-1024 cores shared-memory FPGA architecture that can be used to implement such SoCs. Such an architecture will enable to not only speed up computations but also implement the SOC's components in pure software mode.

Typically SoCs contain several processing cycles wherein input signals from sensors are processed and output signals are generated. The units in these processing cycles may be special hardware circuits, con-

¹Extension of Conference Poster FPGA 2014. Almost all of the material in this paper has not been published in FPGA'14 including architecture details, ideas and all the experiments.

trollers, memories, dedicated CPUs and general purpose CPUs. All these units communicate by point-to-point channels or busses and must maintain timing restrictions. Figure 1 (left) illustrates such a processing cycle for a voice recognition system. The different units $U1 \dots U8$ apply signal processing algorithms such as FIR filter, FFT, Viterbi decoding etc. Such a system would include at least one bus several hardware circuits, memories and few cores. A different approach is to implement this voice recognition system as a parallel shared memory program wherein units of the SOC are executed as parallel algorithms (as described in figure 1 right). The communication between the units will be done via shared data-structures such as parallel queues and shared flags. By increasing the amount of cores allocated to each unit, we can increase its processing rates thus allowing the system to maintain desired time constraints. Furthermore, the system is now implemented in pure software mode thus reducing the development effort and simplifying debugging/testing. The goal is thus to replace current SOC designs that combine heterogeneous cores and hardware accelerators by a set of many homogeneous light cores communicating through shared memory. Note that the proposed architecture implements a close model to the well known PRAM model for parallel computations allowing the use of the vast set of PRAM algorithms that has been developed.²

In order for such a shared memory architecture to be used in a SOC it must satisfy the following requirements:

- Support large number of cores in a single chip/FPGA so that each unit of a processing cycle can be sped up to meet necessary time constraints.
- The clock latencies should be about the same as current SoCs.
- It should not consume extra power compared to current SoCs.
- Shared memory accesses must be made in very few clock cycles since the shared memory is used

²The term “close” is due to the fact that memory references may suffer unexpected delays thus the proposed architecture only approximates the synchronous steps of the classic PRAM.

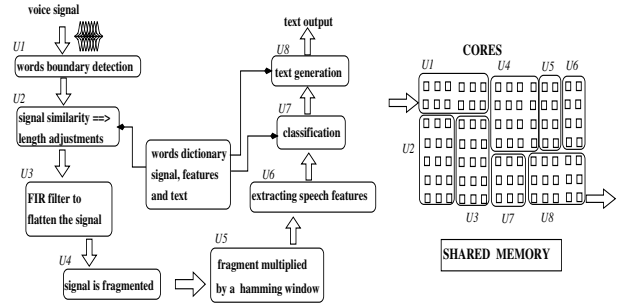


Figure 1: Processing cycle of Voice recognition and its embedding in a shared memory multicore architecture.

for both the communication between the SoC’s units and the parallel processing inside each unit.

However, current shared memory multicore chips support only a relatively small number of cores since shared memory is implemented via cache coherency protocols [14]. This form of shared memory simulation is not scalable and is subject to large delays needed to propagate updated consistent values of shared variables between the caches and the main memory. A more direct way to build shared memory called DMM (Distributed Memory Machine) was proposed as a theoretical implementation for the PRAM model [8]. The DMM uses a multistage network to transfer read/write packets to a set of memory modules each holding a distinct part of the shared address space. This suggests an alternative way to build very large manycore chips where a DMM shared memory replaces the use of cache-based shared memory. Although the theoretical construction of PRAM is well defined, it is not clear how and if at all it will practically function for a high number of cores in a SoC mode. Thus in this work, we study the usefulness of the DMM solution for building 2-1024 manycore FPGA architecture which, as indicated before, can form an alternative architecture for SOC’s.

It is important to understand how, theoretically, shared memory of a multicore machine can work. In the proposed theoretical construction, the shared address space is partitioned to n memory modules. Each core can access any module via a multistage communication network (such as a butterfly network

(BN) or a hypercube). Every global address can be resolved to a module + internal address in that module. Routing the memory requests to the suitable module and back is done through a communication network. The memory space is hashed so that the probability of n load/store instructions executed by n cores to issue more than $\log n$ memory addresses that have been mapped to the same memory module is polynomially small. Thus the fact that each memory module can serve only one core at a time is not likely to delay the execution of these n instructions significantly. This hashing of memory addresses is achieved by using a hash function $h(x)$ pre-selected randomly from a finite family of universal hashing functions $H = \{h_1(x), \dots, h_k(x)\}$ [4]. Each memory reference to an address x is directed (by all cores) to a different address $h(x)$. Intuitively a random selection of $h(x) \in H$ before the program begins, guarantees that the probability of directing more than $\log n$ shared-memory references onto the same module, is polynomially small in the number of cores/memory-modules n . This result is further extended for $\log n$ -wise independent hashing functions guaranteeing that the queue size at a given module is no greater than $\log n$. If the queue size at a memory module is found to be too small during execution, then a different hash function is chosen and the memory is rehashed. This can take some time but the probability of rehashing is low and decreases asymptotically as the length of the program increases. Implementing this theoretical scheme fully may lead to a non-practical design. Thus in this work we study the following main issues:

- How is the memory latency affected by increasing n (number of cores/memory-modules). Practically we need this latency to remain about the same even when the number of cores increases.
- Does the construction work for small values of n or is it valid only for large values of n (e.g., $n = 512, 1024$)?
- How frequent if at all, do we really need to use the memory re-hashing mechanism? Can a small and simple enough family H be used?
- What is the expected latency of a memory reference due to multiple memory references that are

directed to the same memory module? We hope to show that this delay is very small.

- What is the effect of collisions of packets in the BN-switches (congestion)? Since the use of universal hashing is similar to using random destinations, we hope to show that the penalty of packets congestion in the BN is limited. Though theoretical solutions to eliminate BN collisions do exist, they are too complicated to be realized. Thus it is important to see that practically there is no need to use them. In this case we can eliminate the need to use buffer-queues in the BN-switches and even run the BN as a pure combinatorial circuit.
- The BN requires increased wire area of n^2 which can also affect the execution frequency. We hope to show that this is not the case and the internal routing mechanism of the FPGA absorbs this complexity.

A positive answer to these questions implies that the theoretical DMM can be used in a very simple and basic mode for SOCs over FPGAs.

Implementing SOCs as many cores working in parallel can also help reduce power consumption. Power is proportional to *voltage*² (V^2), hence it can be significantly reduced if we are able to reduce the voltage V . However reducing V requires a lower execution frequency. In a many core system, the performance loss caused by the lower execution frequencies can be compensated by the increased parallelization. For example, assume that a given task executed by one fast-core at a given voltage V and frequency f is split to k cores. Ideally, it is now possible to let each of these k cores run at voltage V/k and at frequency f/k still maintaining the original execution time of the fast-core. However, due to the quadratic effect of lowering the voltage, the power consumption of these k cores is $1/k$ of the power consumption of the fast-core. Thus using 2-1024 cores at a slower frequencies may lead to reduced power consumption.

2 Proposed architecture

In this work we study a practical version of this theoretical construction. This architecture does not use

caches, rather relaying on the ability to route memory requests to the suitable memory modules in one or few clock cycles. On the other hand, we use a butterfly network (BN) whose complexity may prevent the low memory latencies that are needed for a practical use of this type of architecture. In particular, the 2D layout of the BN requires an order of n^2 area of wires to connect between its $n \cdot \log n$ switches thus potentially incur long delays. One option is to realize the BN as a multi-stage network wherein packets are stored in the switches and advance to the next level every clock cycle. Another possibility is to implement the BN as a pure combinatorial circuit. The use of multi-stage network where in packets advance to the next level every clock cycle may potentially lead to small latencies since the clock frequency counts only for the latency of one switch. However, it will take $\log n$ steps to route packets through the BN which violates our need for one (or very-few) clock cycle memory latencies. The other option of using a pure combinatorial realization of the BN seems like a better option. In particular, there is a possibility that the complexity of the BN will be absorbed by the FPGA reconfigurable infrastructure and thus the routing complexity of the BN will not affect the memory latencies. This is because FPGAs have already a network which is a crossbar of buses that is used to dynamically interconnect between the different hardware components (such LUTs, registers,...) of the FPGA. Thus, the FPGA may be able to embed the BN without incurring increased latencies. We have built such a multi-core architecture and verified that the FPGA's infrastructure can indeed absorb the complexity of the BN.

The proposed multicore contains the following components (as depicted in figure 2):

- Programming mode is SPMD where each core executes one thread that is loaded and executed from a pre initialized memory module that is local to the core. Though this is a simplified mode of programming it is sufficient for demonstrating the usefulness of such multicores to embedded systems. A simple compiler for C + ASM is used to automatically compile SPMD C programs to n cores.

- A set of $n = 2^k$ $k = 0, \dots, 10$ memory modules $M_1, \dots, M_n$ form the memory shared address space of n Pacoblaze cores (A clone of Xilinx Picoblaze). These are light FPGA soft CPUs that have been modified to support 32 bits (values and memory addresses) and fixed point arithmetics. Each core has call/return stack and an instruction ROM that are managed through local memory modules.
- Memory references are generated as packets where each packet contains the following fields:

$[R/W - bit \mid memory - module \mid internal - address \mid priority]$

- A multistage butterfly topology network (BN) connecting the n memory ports of the cores to the shared memory modules. The BN works as a combinatorial circuit allowing packets to be routed to their destination in one clock cycle (unless collided with another packet). The use of a BN as a combinatorial circuit is thus done in order to save wire area compare to the ability to use a circuit that selects for every memory module the next packet that is destined to it. Thus the whole point of using a combinatorial BN-circuit is to reduce the wire/MUX complexity if we directly connect each core to each memory module. Figure 3 illustrates collision of two packets attempting to use the same exit of a switch where one sent from core 0010 to module 1010 and the other from 1011 to 1000.

When a packet is dropped at a given switch the core that issued this memory reference is blocked and the memory request is repeated in the next clock cycle until the packet reaches its destination. Note that the failing signal is not propagated directly to the core but through the backward BN's switches. Thus the core simply waits until the acknowledge-packet is received at the suitable port of the backward BN. We thus skip the optimization not to wait for the completion of store operations as this would prevent sequential consistency [3]. For example, If two cores execute the program

core-1 : $x = 2; y = 1;$ *core-2* : $while(y == 0); print(x);$

and the initial values are $x = y = 0$ then under sequential consistency the program can not print $x = 0$ which can happen if core-1 will not wait for write-ack.

- When a packet pa is dropped in a switch $S_{i,j}$ it is stored in $S_{i,j}$ in an intermediate-register and will continue in the next clock cycle, not from the original core that issued it, but from $S_{i,j}$. This complicates the logic at each switch but, as depicted in figure 3, can: a) Make free the switches in the path of pa to $S_{i,j}$ so that they can be used by other packets (e.g., pes in the figure). or b) Prevent a possible “left blocking” of pa at the next clock cycle by a packet pc with a higher priority. Collisions (as depicted in figure 3) can now occur not only between two packets entering the BN’s but also with the packet that resides in the intermediate register. In this case (three packets collide) one of the packets is dropped back to the core that issued it, one stays in the intermediate register and one continues to the next level.
- The decision which packet is dropped and which continues is made based on a “priority” of the packets arriving to the switch. We considered the following possibilities:
 - (1)- No priority, e.g., drop the packet sent from the left entry of the switch.
 - (2)- Prefer packets that where sent from smaller core-ids;
 - (3)- Prefer older packets (using time tags);
 - (4)- A packet starts with priority=0 and its priority increases every time it collide and is dropped.
 - (5)- Combination of (4)-(3)-(2) by this order.

We selected option (4) which is the most efficient in terms of latency.

- The theoretical shared memory uses universal hashing scheme [13] to re-map memory addresses so that, with high probability, the number of parallel load/store accesses destined to the same memory module is less than $\log n$. The theoretical framework requires that one hash-function should

be selected in random from a family of n hash functions before a program is executed. This is not practical, we have tested a smaller family of such functions that can be realized without increasing the clock latency or add extra clock cycles to load/store operations.

- Each Pacoblaze core has an I/O-port however these ports can not be left as external ports due to the large number of cores. The cores’ I/O-ports are connected through a bus to an external I/O device (e.g., Ethernet card, Camera or a cellular modem) that can broadcast/collect packets of data to/from local I/O buffers of each core ($I/O - B_0, \dots I/O - B_7$ in figure 2). Each core has a controller that collects the data items of a data-packet that is sent to it by the I/O device or outputs data-items to the external I/O device. The fact that I/O is handled through a local memory at each core, reduces the amount of shared memory transactions needed otherwise. Note that the external I/O device should be able to direct data-packets to specific cores. A control line is used to indicate that the core is not ready to accept another packet and that the I/O device should send it to another core. The I/O device can use the control to indicate that the bus is not free to write packets. Note that it is possible to ignore these control signal and let the I/O device transfer packets at a fixed rate assuming that $\#cores \cdot local_processing_time$ is sufficiently large to allow it to send packets to cores in a fix-rate. Thus the rate in which data/packets can be injected into the chip scales-up with the number of cores which is an important factor for SOC applications that need to process a stream of incoming data in high rates.

Figure 2 illustrates the proposed architecture for $n = 8$ cores ($C_0 \dots C_7$), showing the BN (forward and backward), partition of the shared address space to memory modules, the hashing units, the ring-buffer with two real ports and the interrupt-buss.

It is important to note that the packet lost at the BN does not hamper sequential consistency since the three conditions of sequential consistency [3] are trivially preserved:

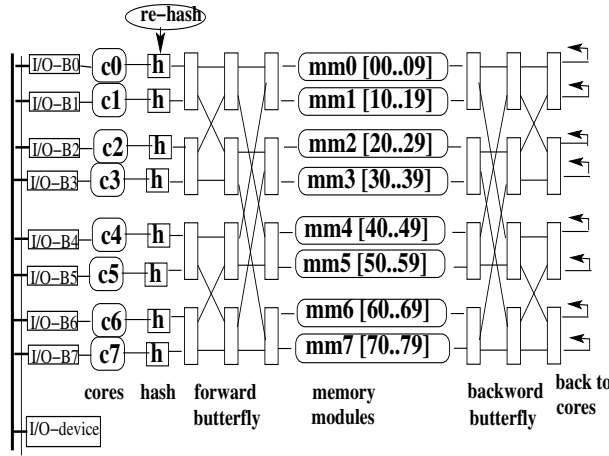


Figure 2: Schematic layout the PRAM multicore architecture.

- Cores issue memory operations in program order as the Pacoblaze cores are not out-of-order and so is the compiler.
- Each core waits (through the backward BN) for every store operation it has started to complete before issuing another operation.
- A load operation L by $core_i$ that returns a value from of a store S performed by $core_j$ can not terminate before $core_j$'s store terminated. This holds because the acknowledge/return-value of L will arrive at-least one clock cycle after the acknowledge of S will arrive to $core_j$. Note that the use of intermediate registers at the switches does not damage the sequential consistency.

It trivially follows that there is no starvation and the priority mechanism that was implemented creates a fair execution since packets that collide and are dropped (back to the core or to an intermediate register) acquire a higher priority.

3 Experimental results

First we show the synthesis results of the proposed architecture for the FPGA using Xilinx's ISE 13.1. The results of the FPGA synthesis of this architecture are depicted in table 1 showing that the clock latency

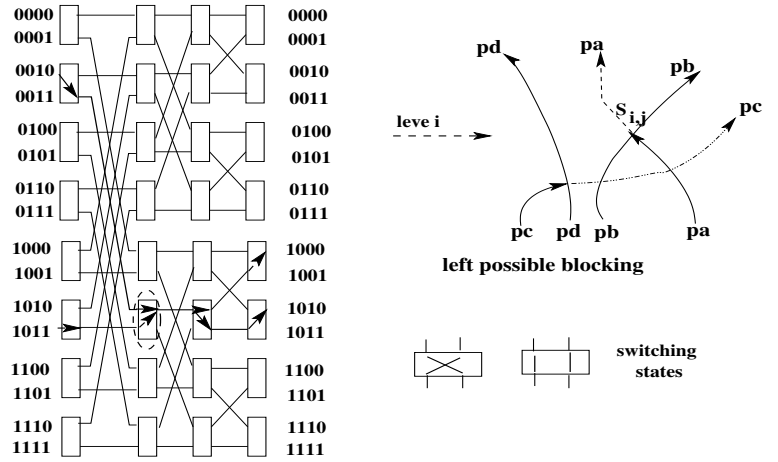


Figure 3: Butterfly network+collision and avoiding collisions due to intermediate restarting of packets.

does not increase significantly when more cores are used. The fact that the resources (LUTs+Registers) at most doubles every time we double the number of cores prove that the connections between the BN-switches is absorbed by the FPGA. It thus follows that the BN can be embedded by the routing infrastructure of the FPGA without increasing the memory latency. Though there is a small decrease in the clock frequency when more cores are used it is very slow compare to the double increase in the number of cores. Clearly resources and power grow proportionally in the number of cores. $n = 64$.

cores	Slice LUTs	Slice Registers	DSP	Dynamic power (W)	Logic power (W)	freq. Mhz
1	872	118	4	0.04	0.004	101
2	1854	237	8	0.082	0.009	101
4	4350	482	16	0.168	0.02	99
8	8621	979	32	0.34	0.042	98
16	16785	1988	64	0.681	0.088	96
32	37274	4037	128	1.416	0.193	95
64	81216	8198	256	2.92	0.418	95
128	171731	16647	512	5.964	0.901	95
256	363975	33800	1024	12.189	1.903	87
512	756602	68617	2048	25.003	4.069	86
1024	1813814	139274	3568	54.286	9.941	83

Table 1: Synthesis results for the FPGA Virtex-7 and ASIC.

Next we show the speedup and effect of collisions (at the memory modules and at the BN's switches) for several parallel programs. These results are given for the Reverse-LSB hashing function that achieved maximal speedups for all programs. A more detailed results showing the effect of using different hash-

function will be given next for the Viterbi decoding algorithm. The results Table 2 show high speedups and average of 1-2 clock cycle per memory reference.

program	intermediate register	min load	max load	cycles / conflicts	speedup
odd-even merge sort	no	304225	648205	6.84	480.60X
Matrix multiplication	no	2445329	2457127	443.06	1019.08X
sum	no	9384	473474	1.18	20.30X
N-body	no	1406618	1815890	14.59	547.70X
Prime numbers	no	64519	87931	36.28	751.36X
List ranking	no	74554	852179	1.02	1024.00X
Viterbi	no	239957	308311	10.85	796.97X
Bloom filter	no	29895	61645	5.11	496.59X

Table 2: Speedup/collisions for parallel Reverse-LSB of several parallel programs (1024 cores).

Table 3 shows the results of running a parallel implementation of the viterbi algorithm using a 1024-core configuration. This algorithm is used in decoding the convolutional codes used in both CDMA and GSM digital cellular networks, in addition to various other types of wireless networks. The results indicate that the Reverse LSB function gave the best speedup, 796.97X without the intermediate register and 818.21X with the register while maintaining a good conflict rate of 1 conflict per 10.85 and 12.18 cycles respectively. The Reverse MSB function obtained the worst results both speedup wise and cycles/conflict wise: 118.94X and 1.25 respectively with the intermediate register disabled. Recall that in Viterbi algorithm we have an automata A with probabilities on its state transitions (called inner transitions) and probabilities of observing a set of output symbols from each state. The goal is, given a sequence of observable outputs $X = x_1, \dots, x_k$ to find the most likely sequence of inner transitions that produced the given sequence of observable outputs (i.e., the sequence with the highest accumulating probabilities). In order to compute this sequence we build a “trellis” of all possible state-transitions where each level i in the trellis contains the accumulate probabilities of all possible inner transitions that can produce the prefix $x_1, \dots, x_i$. Note that in this experiment we used a large hidden automata with 1024 states, thus there were 10^6 transitions between consecutive levels of the trellis and hence a large number of computations that can be done in parallel.

Finally we evaluated (table 4) the effect of the priority mechanisms described in section 2) for prime

Hash-func	intermediate register	min load	max load	cycles / conflicts	speedup
LSB	no	239959	426420	4.51	576.23X
Reverse LSB	no	239957	308311	10.85	796.97X
MSB	no	239957	2002621	1.28	122.70X
Reverse MSB	no	239957	2065937	1.25	118.94X
Middle Bits	no	239957	1180715	1.54	211.04X
Reverse Middle Bits	no	239957	1151426	1.54	213.40X
Circular Rotate Left	no	240020	1038006	1.68	236.72X
Reverse Circular Rotate Left	no	240020	957105	1.78	256.73X
LSB	yes	239959	378979	5.43	648.36X
Reverse LSB	yes	239956	300307	12.18	818.21X
MSB	yes	239957	2025312	1.25	121.32X
Reverse MSB	yes	239957	2025312	1.25	121.32X
Middle Bits	yes	239957	1164324	1.54	211.04X
Reverse Middle Bits	yes	239957	1152715	1.54	213.16X
Circular Rotate Left	yes	240020	1013257	1.71	242.50X
Reverse Circular Rotate Left	yes	240020	955576	1.78	257.14X

Table 3: Speedup/collisions for Viterbi algorithm. 1024cores

numbers, sorting and nbody (using the Reverse LSB hash-function). Note that all previous results were obtained with the no-priority scheme (1). All of the priority mechanisms improved the speedups e.g., a speedup of 1002.19X for primes using core-ids compare to 751.36X with no priority. The sorting program used barrier locks between iterations for which the core-ids scheme caused starvation (the ----- result in table 4). The nbody measurements were applied to the program after all locks have been removed so that the core-id scheme will not cause starvation. Basically these results suggest that using the number of drops priority is a good choice since it is simple to implement, starvation-free and obtained good results.

priority mechanism	speedup primes	speedup sorting	speedup nbody
1 (none)	751.36X	480.60X	602.59X
2 (code id)	1002.19X	---	582.82X
3 (time stamp)	993.09X	560.27X	667.49X
4 (number of drops)	993.09X	560.27X	667.49X
5 (combo of 2-4)	991.68X	559.93X	668.12X

Table 4: Effect of different priority mechanisms on prime numbers and sorting.

In conclusion, the above experiments indicate that shared memory 1K-core chips can be implemented in the proposed manner as follows:

- Most of the algorithms gave good speedups and good scalability.
- It seems that two hashing functions (LSB and Reverse LSB) suffice for all benchmarks. This because most algorithms traverses arrays/list ar-

ranged linearly in the memory the Reverse LSB simply distributed these data-structures evenly between the memory modules such that consecutive addresses were now mapped to different memory modules. This reduced the amount of collisions at the BN switches and at the memory modules to a minimum.

- The experiments assert the usefulness of the intermediate register technique which is fairly simple and do preserve the one cycle latencies of the BN (the BN basically remains a pure combinatorial circuit). We remark that we have implemented a BN with waiting buffer-queues at every node. However, using such a BN increased the amounts of resources and latencies significantly up to a point where large number of cores could not be used.
- Seems that routing complexity of the BN has no significant effect for the FPGA realization making attractive for FPGA realizations.

4 Related works

The theoretical construction of DMM to simulate PRAM was studied in several works: [15] proposed a PRAM DMM architecture that use universal hashing not only to eliminate congestions at the memory modules but also eliminate congestion at the communication network that connects the cores to [8] the memory module. This method used a complex scheme imposing a monotone order for transferring memory references between the switches of the communication network. The SB-PRAM [9] is an attempt to implement Ranade’s scheme in real hardware. However, as indicated, we hope to show that congestion in the communication network will practically not affect the performance even when Ranade’s scheme is not used.

The idea of building a PRAM (Parallel Random Access Model) on a single chip, where the chip include the cores, memory-modules and the communication network) was proposed and to some extent implemented in [1] by U. Vishkin [16]. Different types of CPU-to-Memories interconnection networks have

been studied and compared. The architectures presented do not include resource-sharing bus, which is similar to the architecture we choose to implement. [16] proposes an FPGA-based system and exploits the explicit Multiple-Threading programming model. Next [5] proposed a hardware architecture suitable for performing one of the main tasks of 21-st century’s challenge of information society, namely the system able to perform parallel computing efficiently. This architecture uses a collection of symmetric (non-dedicated) processors working with the distributed memory blocks (DMM) while the memory is shared from the programmer’s point of view. One of the goals of this architecture is to simplify the programming model by using a very complex hardware Scheduler Unit, while we propose simpler hardware solution by off-loading some software engineering tasks to the compiler implementing SMT and using the dedicated ISA for this purpose.

Unfortunately, by definition, the bandwidth of a BN is restricted. Shared-memory cannot be simulated on a BN while still retaining optimal efficiency (optimal efficiency is where the time-processor product is within a constant ratio of the sequential version). Although the latency from the routing delay across the network can be hidden with parallel slackness (multiple PRAM processors are simulated on one physical simulating processor), network congestion delays cannot. If requests are initiated at times 0 and 1 then a network delay of L will lead to replies in L and $L+1$ but if the delay L is caused by congestion then the replies will come in L and $2L$ time. However while memory requests are limited in number contention can be restricted using hashing.

In turning our proposed SoC into ASIC, there’s a requirement for large amounts of on-chip memory. this is achievable with a three-dimensional integrated circuit [2] combining the PRAM in a 3D-stacked memory fashion with a many-core design [6]. Current developments in the field of memristors promise memristor arrays built on a CMOS chip [10], while others showed the benefits to bandwidth and latency of stacking DRAM on-die [7], as such, we strive toward an ASIC design combining a many-core architecture with on-die PRAM.

It is also important to note that a drive towards

implementing, in software, parts of the SoC that are traditionally done in hardware, such as a modem or a radio, and running them on a custom multi-core chip is something already being done in latest commercial chips such as the case in NVIDIA's software defined radio technology [12]. Unlike the current NVIDIA implementation, which uses 8 cores and lacks a huge on-chip shared memory. our proposed chip provides a much larger amount of (simpler) cores connected to a shared memory system that can be up to few gigabytes in our current 32-bit implementation

It is important to compare PRAM on a chip to GPGPU architectures and in particular to Fermi [11] which contains 512 "cores" supporting a small global shared memory. Unlike PRAM on a chip or Multicores, GPGPUs works best in single instruction multiple data (SIMD) mode supporting efficient execution of scatter/gather load/store instructions. This is because only if data addresses produced by the GPU's cores fall in the same DRAM/CACHE row/line then low memory latencies can be obtained. This also contains high entry overhead from the end-programmer and requires following complicated rules. In PRAM on a chip or multicore there is a uniform cost of all memory references made by the different cores even for arbitrary addresses and consequently programming style supports general threads. [?] contains a short indication to a shared memory manycore architecture, however the architecture itself and the experimental results have not been described.

References

- [1] P. Bach, M. Braun, A. Formella, J. Friedrich, T. Grun, and C. Lichtenau. Building the 4 processor sb-pram prototype. In *HICSS '97: Proceedings of the 30th Hawaii International Conference on System Sciences*, page 14, 1997.
- [2] B. Black, M. Annavaram, N. Brekelbaum, J. DeVale, L. Jiang, G. H. Loh, D. McCaule, P. Morrow, D. W. Nelson, D. Pantuso, et al. Die stacking (3d) microarchitecture. In *Microarchitecture, 2006. MICRO-39. 39th Annual IEEE/ACM International Symposium on*, pages 469–479. IEEE, 2006.
- [3] D. E. Culler, A. Gupta, and J. P. Singh. *Parallel Computer Architecture: A Hardware/Software Approach*. Morgan Kaufmann Publishers Inc., 1st edition, 1997.
- [4] M. Dietzfelbinger and F. Meyer auf der Heide. A new universal class of hash functions and dynamic hashing in real time. *Automata, Languages and Programming*, pages 6–19, 1990.
- [5] R. Ginosar and N. Bayer. Tightly coupled multiprocessing: Architectural and technical challenges.
- [6] K. D. Hyun et al. 3d-maps: 3d massively parallel processor with stacked memory. In *Solid-State Circuits Conference Digest of Technical Papers (ISSCC), 2012 IEEE International*, pages 188–190. IEEE, 2012.
- [7] D. Jevdjic, S. Volos, and B. Falsafi. Die-stacked dram caches for servers: hit ratio, latency, or bandwidth? have it all with footprint cache. In *Proceedings of the 40th Annual International Symposium on Computer Architecture*, 2013.
- [8] R. Karp, M. Luby, and F. Meyer auf der Heide. Efficient pram simulation on a distributed memory machine. *Algorithmica*, 16(4):517–542, 1996.
- [9] J. Keller, W. Paul, and D. Scheerer. Realization of prams: Processor design. *Distributed Algorithms*, pages 17–27, 1994.
- [10] K. Kuk-Hwan et al. A functional hybrid memristor crossbar-array/cmos system for data storage and neuromorphic applications. *Nano letters*, 12(1):389–395, 2011.
- [11] J. Nickolls and W. Dally. The gpu computing era. *Micro, IEEE*, 30(2):56–69, 2010.
- [12] NVIDIA. Nvidia software defined radio modem technology, the modem innovation inside nvidia i500 and tegra 4i. *NVIDIA Whitepaper*, 2013.

- [13] A. Ostlin and R. Pagh. Uniform hashing in constant time and linear space. In *STOC '03: Proceedings of the thirty-fifth annual ACM symposium on Theory of computing*, pages 622–628. ACM, 2003.
- [14] M. S. Papamarcos and J. H. Patel. A low-overhead coherence solution for multiprocessors with private cache memories. *SIGARCH Comput. Archit. News*, 12:348–354, 1984.
- [15] A. Ranade. How to emulate shared memory. *Journal of Computer and System Sciences*, 42(3):307–326, 1991.
- [16] X. Wen and U. Vishkin. Fpga-based prototype of a pram-on-chip processor. In *CF '08: Proceedings of the 5th conference on Computing frontiers*, pages 55–66, 2008.